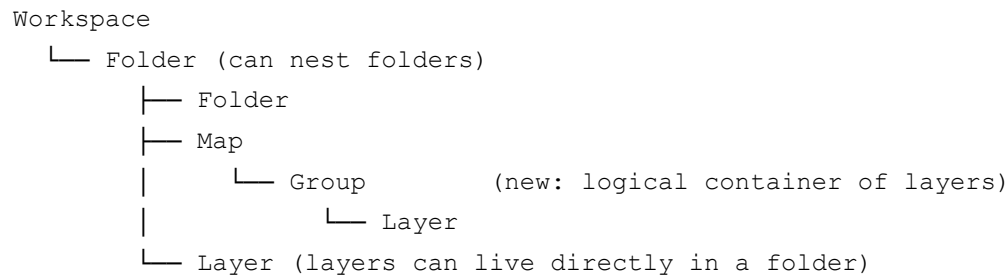


Geo Platform Permission System — Design Document (v2)

1. Overview

Hierarchical permission system for geographic data, covering five resource levels:



Two orthogonal concepts govern access:

1. **Role-based permission** — what a user can *do* on a resource (Admin / Manager / Editor / Viewer), assigned to individual users or to **Teams**.
2. **Visibility** — whether a user can see a resource exists at all. Visibility is whitelist-only everywhere, all the way up to the workspace level. There is no “public within the system” default — every resource is invisible until a user has an explicit grant on it or on something beneath it.

No “deny” semantics exist anywhere in this system — only grants. Removing access always means removing a grant (or removing someone from a Team), never adding a block.

2. Roles

2.1 System-wide roles

Role	Capabilities
SuperEditor	Full control over everything in the system: create workspaces, edit any data, grant/remove any permission anywhere. Bypasses all visibility/restriction checks — never needs to be explicitly whitelisted on anything.

SuperViewer	Can view everything in the system. Cannot edit anything or change any permission. Also bypasses visibility checks (sees everything), but read-only.
--------------------	---

Only a **SuperEditor** can create a new workspace, and they are automatically granted Admin on it (no separate whitelist step needed).

2.2 Per-resource roles

These apply independently at each level (workspace, folder, map, group, layer). A user's role on a resource is scoped to that resource (and whatever it cascades down to), not global.

Role	Can grant/remove	Can edit content	Can view
Admin	Admins, Managers, Editors, Viewers — full control over permissions and visibility on this resource	Yes	Yes
Manager	Editors, Viewers only (cannot touch Admins or other Managers)	Yes	Yes
Editor	No	Yes (within their granted scope)	Yes
Viewer	No	No	Yes

Rules of thumb:

- Admin is the only role that can create/remove other Admins and Managers.
- Manager can add/remove Editors and Viewers (individual users or Teams), but never Managers or Admins.
- Role assignment must be validated server-side in the resolver (§6) — never trust the client to enforce "Manager can't promote to Admin."

3. Resource Hierarchy & the New "Group" Level

A **Group** sits between Map and Layer. It's a logical container for organizing layers within a map (e.g., "Transportation," "Administrative Boundaries"). Groups behave like every other resource in the tree for permission and visibility purposes — they can hold their own role grants, their own visibility whitelist, and can independently break inheritance/restriction

from their parent map.

```
Workspace → Folder → Folder → Map → Group → Layer
           ↳ Layer (folders can also hold layers directly)
```

Each resource type's children:

- **Workspace:** Folders, Maps, Layers (directly).
 - **Folder:** Folders, Maps, Layers.
 - **Map:** Groups, Layers (a layer can sit in a map without being in a group).
 - **Group:** Layers only.
-

4. Visibility & Restriction Model

This is the core addition in v2. It governs *whether a user can see a resource exists*, independent of what role they'd have if they could see it.

4.1 The fundamental rule

Everything is invisible by default. A user sees a resource if and only if:

- They have an explicit grant (direct or via Team) on that resource, **or**
- The resource is an ancestor of something they have an explicit grant on (i.e., it's part of the **path** to something they can access).

This applies uniformly from the workspace level all the way down to layers. There is no separate "restricted" flag needed at the workspace level — workspaces are always whitelist-only. The "restricted" toggle becomes relevant starting at the folder level, where it controls whether a subtree carves itself out from the broader access already granted at an ancestor level.

4.2 How restriction works at folder/map/group level

- When you create a folder, map, or group, you can mark it **restricted**.
- **Restriction cascades down automatically and irreversibly downward at creation time:** if a folder is restricted, every map, group, and layer created inside it is restricted by default. There is no toggle to make a *new* child unrestricted while the parent is restricted — restriction always propagates down.
- However, **restriction is decided independently per resource going forward** —

meaning each resource has its own whitelist of users/Teams once it's restricted, and an Admin can explicitly add people to any specific level (folder, map, group, or layer) without those people needing to be on a parent's whitelist.

- Example: Workspace A is visible to Team A (everyone in Team A has a role on the workspace). Inside it, Folder A is marked restricted with its own whitelist that does *not* include all of Team A. A member of Team A who isn't on Folder A's whitelist cannot see Folder A at all, even though they have access to the workspace it lives in.

4.3 Path-based visibility (the key behavior)

If a user has an explicit grant somewhere **inside** a restricted subtree, but isn't on the whitelist of one or more of its ancestors, they don't get blocked — instead, they see the **minimal path** down to the thing they have access to, and nothing else:

- They see the names of the ancestor folder(s), map, and group needed to navigate to their resource.
- They do **not** see sibling resources at any level along that path (other folders, other maps in the folder, other groups in the map, other layers in the group).
- This is true at every level, including the workspace itself: if a user has permission only on a layer inside Workspace B, but no grant on Workspace B itself, they still see Workspace B appear in their workspace list — purely as the path to that layer — without seeing anything else inside it.

Concrete example (matches the walkthrough during design): Workspace A contains Folder A (with Map A inside it) plus sibling Folders B, C, D. A user is granted permission only on Map A. When they open the system:

- They see Workspace A.
- They see Folder A (because it's on the path to Map A) — but not Folders B, C, or D.
- They see Map A, and whatever they're permitted to see inside it.

This filtering applies at every level simultaneously — it's not "unlock the folder, then apply normal rules below." The tree shown to a user is always the union of every path leading to something they have an explicit grant on.

4.4 Admins, Managers, and visibility

- **Admins and Managers are not subject to path-filtering on resources they administer.** If you're an Admin or Manager on a folder, you see its entire tree, restricted or not — admins/managers can't be blind to what they manage.

- **SuperEditor and SuperViewer bypass all restriction and visibility logic everywhere**
— no whitelist entries are ever needed for them.
-

5. Teams

Renamed from “groups” to avoid collision with the new Group resource type. Teams are the scalable mechanism for assigning the same role to many people at once.

- A Team is a named collection of users (e.g., “GIS-Leads,” “Field Surveyors”). Can be synced from Microsoft Entra ID or created locally.
 - Permissions and whitelist entries can be granted to a Team as a whole, rather than to each member individually — this is the primary way to manage access at scale across hundreds of users.
 - A user can belong to multiple Teams. **If a user has roles from multiple Teams on the same resource, the highest-privilege role wins.**
 - When a user is removed from a Team, they lose whatever that Team granted them **immediately** — the resolver checks live Team membership on every resolution. Any **direct** (personal) grants the user has are untouched and remain independent of Team membership.
 - **Whitelisting** (for restricted resources) works the same way: you can whitelist an individual user or an entire Team. Whitelisting a Team makes the resource visible to all its members, but — as with any grant — it’s always made *with a role attached* (e.g., “Team A → Editor on this Map”). There’s no such thing as visibility without a role; granting visibility and granting a role happen in the same action.
 - **Team search:** any Manager or above can search the full list of Teams in the system (not filtered by what that Team already has access to) when deciding who to add to a resource.
-

6. Permission Transparency (“Why do I have this access?”)

Two related features, both built on the same underlying breakdown logic.

6.1 “My Access” — self-service view

When viewing any resource, a user can see a breakdown of their own effective permission on it:

- Every source contributing to their access: direct grant (if any) and every Team they belong to that has a grant on this resource.
- Listed **ordered from highest to lowest privilege**, with the winning (effective) role clearly marked.
- Example: *Effective role: Editor. Sources — Personal grant: Editor. Via Team "Field Surveyors": Viewer.*

6.2 Manager/Admin lookup — searching others

Any Manager or Admin, on any resource they manage (workspace, folder, map, group, or layer), can:

- Search for a specific **user** or **Team** and see their effective permission on that resource.
- If the result comes from multiple Teams, see **each Team's contribution listed separately, highest first** — same breakdown format as §6.1, just looking at someone else.
- This search is not restricted to people who already have access — a Manager can search any user or Team in the system and get a clear "no access" result if nothing applies.
- Managers/Admins doing this lookup always see the full tree and full permission picture — they are not subject to path-filtering themselves (§4.4).

7. Permission Resolver

The single source of truth for "can user X do Y on resource Z, and can they even see it." Every API endpoint must call this — no permission or visibility logic should live anywhere else.

7.1 Resolution algorithm (role + visibility combined)

```
function resolve_permission(user_id, resource_id):
    if user.is_superEditor: return ADMIN    # full rights, full visibility, everyw
    if user.is_superViewer: return VIEWER  # read-only, full visibility, everywhe

    cached = redis.get(f"user:{user_id}:resource:{resource_id}:permission")
    if cached: return cached

    direct_roles = get_direct_grants(user_id, resource_id)
    team_roles   = get_team_grants(user_id, resource_id)    # live team membership
    explicit_roles = direct_roles + team_roles
```

```

if explicit_roles:
    result = highest(explicit_roles)
elif resource.inherits_from_parent and not resource.breaks_inheritance:
    result = resolve_permission(user_id, resource.parent_id)
else:
    result = NONE

redis.set(f"user:{user_id}:resource:{resource_id}:permission", result, ttl=..)
return result

function get_visible_tree(user_id, root_resource_id):
    # Returns only resources the user has an explicit grant on,
    # plus every ancestor needed to form a path down to each of them.
    explicit_resource_ids = get_all_explicit_grant_resource_ids(user_id) # direc
    visible_ids = set()
    for rid in explicit_resource_ids:
        visible_ids.add(rid)
        visible_ids.update(get_ancestors(rid)) # walk up to root
    return build_filtered_tree(root_resource_id, visible_ids)

```

Admins/Managers calling this for *their own* navigation, on a resource they administer, skip the filtering in `get_visible_tree` and just receive the full subtree.

7.2 Key properties

- **Single code path.** No endpoint should ever do its own ad hoc permission or visibility check. They all call into this resolver.
- **Highest-wins**, across direct grants and every contributing Team.
- **Visibility is path-inclusive, breadth-exclusive** — ancestors of a granted resource are visible; siblings are not, at every level.
- **Restriction cascades down at creation**, but each resource's own whitelist is independent thereafter.

8. Performance & Caching Strategy

At hundreds of users, Redis + indexed Postgres is sufficient — no need for materialized permission tables.

8.1 Caching

- Cache resolved role per user per resource: `user:{user_id}:resource:{resource_id}:permission`.
- Cache the visible-resource-id-set per user per workspace: `user:{user_id}:workspace:{workspace_id}:visible_ids`, used to build the filtered tree without recomputing ancestor walks on every page load.
- Invalidate only the specific affected keys when a grant, whitelist entry, or Team membership changes — never a full rebuild.
- **Prefetch on navigation:** when a user opens a workspace, compute and cache their full visible-id-set for it in one pass, so opening any folder/map/group within it is a pure cache hit.

8.2 Database

- Index `resources(parent_id)` for ancestor/descendant walks.
- Index `permission_grants(resource_id)` and `permission_grants(grantee_type, grantee_id)`.
- Index `team_members(user_id)` for fast “what Teams is this user in” lookups.
- Use a recursive CTE (see §9) to compute ancestor paths and filtered trees in a single query rather than N round trips.
- Keep the audit log in a separate table with its own indexes so high-volume writes never compete with permission-read performance.

9. Audit Log

A simple append-only log — who did what, when. No restore/rollback, and no special-casing between role grants and whitelist/visibility changes; they’re recorded as the same kind of “permission action” entry.

Each entry records: actor user ID, resource ID, grantee (user or Team), role, action (`grant` / `revoke`), timestamp.

10. Database Schema (PostgreSQL)

```
-- Resource hierarchy (workspace, folder, map, group, layer all share a shape)
```

```

CREATE TABLE resources (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    type              TEXT NOT NULL CHECK (type IN ('workspace', 'folder', 'map',
    parent_id         UUID REFERENCES resources(id),
    name              TEXT NOT NULL,
    inherits_from_parent BOOLEAN NOT NULL DEFAULT TRUE,
    is_restricted      BOOLEAN NOT NULL DEFAULT FALSE, -- whitelist-only mode a
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_resources_parent ON resources(parent_id);

-- Teams (local or synced from Entra ID)
CREATE TABLE teams (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name              TEXT NOT NULL,
    source             TEXT NOT NULL DEFAULT 'local' CHECK (source IN ('local', 'entr
    entra_group_id     TEXT,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE team_members (
    team_id           UUID NOT NULL REFERENCES teams(id) ON DELETE CASCADE,
    user_id           UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    PRIMARY KEY (team_id, user_id)
);
CREATE INDEX idx_team_members_user ON team_members(user_id);

-- Explicit permission grants (to a user OR a team, on a resource)
-- A grant simultaneously establishes role AND visibility/whitelist membership.
CREATE TABLE permission_grants (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    resource_id       UUID NOT NULL REFERENCES resources(id) ON DELETE CASCADE,
    grantee_type       TEXT NOT NULL CHECK (grantee_type IN ('user', 'team')),
    grantee_id        UUID NOT NULL, -- user_id or team_id depending on grantee_typ
    role              TEXT NOT NULL CHECK (role IN ('admin', 'manager', 'editor', 'vie
    granted_by        UUID NOT NULL REFERENCES users(id),
    granted_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_grants_resource ON permission_grants(resource_id);
CREATE INDEX idx_grants_grantee ON permission_grants(grantee_type, grantee_id);

-- System-wide role flags
ALTER TABLE users ADD COLUMN system_role TEXT CHECK (system_role IN ('superEditor

-- Audit log (append-only)
CREATE TABLE permission_audit_log (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

    actor_user_id    UUID NOT NULL REFERENCES users(id),
    resource_id      UUID NOT NULL REFERENCES resources(id),
    grantee_type     TEXT NOT NULL CHECK (grantee_type IN ('user','team')),
    grantee_id       UUID NOT NULL,
    role             TEXT NOT NULL,
    action           TEXT NOT NULL CHECK (action IN ('grant','revoke')),
    occurred_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_audit_resource ON permission_audit_log(resource_id);
CREATE INDEX idx_audit_actor ON permission_audit_log(actor_user_id);

```

Role hierarchy for “highest wins” comparisons (numerically, higher = more privilege):

viewer=1, editor=2, manager=3, admin=4.

10.1 Recursive CTE for ancestor path / filtered tree

```

-- Get the full ancestor path for a given resource (root-first)
WITH RECURSIVE ancestors AS (
    SELECT id, parent_id, name, type, 0 AS depth
    FROM resources WHERE id = :resource_id
    UNION ALL
    SELECT r.id, r.parent_id, r.name, r.type, a.depth + 1
    FROM resources r
    JOIN ancestors a ON r.id = a.parent_id
)
SELECT * FROM ancestors ORDER BY depth DESC;

-- Get every resource id a user can explicitly see (direct + via teams),
-- then expand to their ancestors, in one round trip.
WITH explicit AS (
    SELECT resource_id FROM permission_grants
    WHERE grantee_type = 'user' AND grantee_id = :user_id
    UNION
    SELECT pg.resource_id FROM permission_grants pg
    JOIN team_members tm ON tm.team_id = pg.grantee_id AND pg.grantee_type = 'tea
    WHERE tm.user_id = :user_id
),
visible AS (
    SELECT id, parent_id FROM resources WHERE id IN (SELECT resource_id FROM expl
    UNION
    SELECT r.id, r.parent_id
    FROM resources r
    JOIN visible v ON r.id = v.parent_id
)
SELECT DISTINCT id FROM visible;

```

11. UI Plan

11.1 Navigation

- Left sidebar shows the resource tree, but **filtered per user** per §7.1's `get_visible_tree`: Workspace → Folders → Maps → Groups → Layers, showing only paths the user has access along.
- Admins/Managers viewing resources they administer always see the unfiltered full tree.

11.2 Permission & restriction management panel

- Each item's three-dot menu → **"Manage Permissions"** opens a side panel showing:
 - Current grants (user/Team + role), with Remove buttons restricted by the actor's own role (Manager can't remove Admin/Manager entries).
 - An "Add" control to grant a user or Team a role — restricted similarly (Admin sees all four roles to assign; Manager sees only Editor/Viewer).
 - A **"Restricted"** toggle (available on folders, maps, and groups). Turning it on activates whitelist-only visibility for this resource and everything created under it going forward.
 - When restricted, the whitelist (same list as the grants above — visibility and role are granted together) is managed right here; there's no separate visibility-only control.

11.3 "My Access" detail view

- A **"Why do I see this?" / "My Access"** button on any resource shows the current user's effective role, broken down by source (direct grant, each contributing Team), ordered highest to lowest, with the effective role highlighted.

11.4 Manager/Admin user & Team lookup

- On any resource a Manager or Admin manages, a **"Check Access"** search lets them type a user or Team name and see that entity's effective permission on this resource — same breakdown format as §11.3, with each contributing Team listed separately, highest first. Search covers all users/Teams in the system, not just ones with existing access.

11.5 Team management

- Separate "Teams" admin section: list Teams, view/add/remove members, reverse-lookup which resources a Team has access to.

- Entra-synced Teams have read-only membership in-app (synced automatically); local Teams are fully editable.

11.6 Audit view

- Per-resource "History" tab: reverse-chronological log of grant/revoke actions, filterable by date or actor.
-

12. Open Items / Things to Decide During Implementation

- Exact TTL strategy for Redis permission and visible-id-set cache entries.
- Whether Entra Team sync runs on a schedule or via webhook for near-real-time membership updates.
- Pagination/virtualization strategy for the resource tree UI once folder/map/group counts grow large within a single user's visible set.